Managing Traffic with OpenShift Service Mesh

Lesson | Lab Environment

Guided Exercise: Traffic Routing and Splitting with OpenShift Service Mesh

Implement weighted, header-based, and path-based routing to control traffic distribution between service versions.

Execute a progressive canary deployment by incrementally shifting traffic percentages.

Outcomes

- Route traffic to services in Red Hat OpenShift Service Mesh (OSSM) based on weighted routing, HTTP header, and URI path matching.
- Implement progressive canary deployments with traffic shifting.

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start meshtraffic-routing
```

Also, run the following command to update your `$PATH` variable and make the `traffic_gen.py` command available immediately. Run it once after creating a new environment.

```
[student@workstation ~]$ source ~/.bashrc
```

The `lab start` command performs the following activities:

- Creates the `meshtraffic-routing` namespace.
- Adds the `meshtraffic-routing` namespace to the service mesh.
- Deploys `ratings` and three versions of `reviews` applications in the `meshtraffic-routing` namespace.
- Creates basic gateway and virtual service resources for ingress traffic.
- Configures the `traffic_gen.py` script to generate traffic.

Instructions

1. Verify the initial status of the service mesh.

   1.1. In a new terminal window, log in to the OpenShift cluster as the `developer` user with the `developer` password, then switch to the meshtraffic-routing project:

   ```
   [student@workstation ~]$ oc login -u developer \
   -p developer https://api.ocp4.example.com:6443
   Login successful.
   ...output omitted...
   [student@workstation ~]$ oc project meshtraffic-routing
   ...output omitted...
   ```

   1.2. Verify that the applications are running in the `meshtraffic-routing` namespace.

```
[student@workstation ~]$ oc get pods
NAME                              READY   STATUS    RESTARTS   AGE
ratings-v1-...                    2/2     Running   0          5m
reviews-v1-...                    2/2     Running   0          5m
reviews-v2-...                    2/2     Running   0          5m
reviews-v3-...                    2/2     Running   0          5m
```

All pods should show `2/2` containers ready, indicating that Istio injected the Envoy sidecar proxy.

1.3. Change to the exercise directory.

```
[student@workstation ~]$ cd ~/course/labs/meshtraffic-routing
[student@workstation meshtraffic-routing]$
```

1.4. Verify external access to the mesh by using the `traffic_gen.py` script. Observe that traffic is randomly distributed across all three versions without any routing rules.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/rev
iews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/100] ✅ HTTP 200 -- red (49.9ms)
[2/100] ✅ HTTP 200 -- red (15.0ms)
[3/100] ✅ HTTP 200 -- red (15.5ms)
[4/100] ✅ HTTP 200 -- No stars (12.1ms)
[5/100] ✅ HTTP 200 -- No stars (8.7ms)

...output omitted...

📈 Response Distribution
================================================================================
┌──────────────────────────┬──────────┬──────────────────────┐
│ Response                 │ Count    │ Percentage           │
├──────────────────────────┼──────────┼──────────────────────┤
│ No stars                 │ 42       │               42.0%  │
│ red                      │ 30       │               30.0%  │
│ black                    │ 28       │               28.0%  │
└──────────────────────────┴──────────┴──────────────────────┘
```

Your values might differ.

Notice that Kubernetes randomly load balances traffic across all three versions, resulting in approximately equal distribution. Without OSSM routing rules, basic Kubernetes service load balancing distributes requests evenly across all pod endpoints. In the following steps, you configure OSSM resources to gain precise control over traffic distribution.

> **NOTE**
>
> Remember to run the `source ~/.bashrc` command to update your `$PATH` variable and make the `traffic_gen.py` script available.
>
> ```
> [student@workstation ~]$ source ~/.bashrc
> ```

2. Implement weighted routing to control traffic distribution.

In this step, you implement weighted routing to precisely control what percentage of traffic goes to each version of the `reviews` service. Unlike the random load balancing you observed in step 1, weighted routing gives you deterministic control over traffic distribution.

You configure an 80/15/5 split where:

Weight	Version	Response	Purpose
80%	v1	No stars	Stable production version
15%	v2	black stars	Mature version with established user base
5%	v3	red stars	New version being tested with minimal exposure

2.1. Review the `reviews-dr.yaml` file. It creates a destination rule OSSM resource that does the following:

- Defines the `reviews` service as the target.
- Creates `v1`, `v2`, and `v3` subsets based on pod version labels.
- Enables traffic routing to specific service versions.

```
[student@workstation meshtraffic-routing]$ cat reviews-dr.yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews (1)
  subsets: (2)
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
  - name: v3
    labels:
      version: v3
```

(1) The Kubernetes service name for the `reviews` microservice.

(2) Define subsets for each version of the `reviews` service.

2.2. Create the destination rule OSSM resource.

```
[student@workstation meshtraffic-routing]$ oc create -f reviews-dr.yaml
destinationrule.networking.istio.io/reviews created
```

2.3. Review the `reviews-weighted-vs.yaml` file. It updates the `reviews` virtual service OSSM resource to implement weighted routing, which does the following:

- Routes 80% of traffic to `reviews-v1`.
- Routes 15% of traffic to `reviews-v2`.
- Routes 5% of traffic to `reviews-v3`.

```
[student@workstation meshtraffic-routing]$ cat reviews-weighted-vs.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route:
    - destination: (1)
        host: reviews
        subset: v1
      weight: 80 (2)
    - destination:
        host: reviews
        subset: v2
      weight: 15
    - destination:
        host: reviews
        subset: v3
      weight: 5
```

(1) Define multiple destinations for traffic splitting.

(2) Specify the percentage of traffic for each subset.

2.4. Update the virtual service OSSM resource.

```
[student@workstation meshtraffic-routing]$ oc replace -f reviews-weighted-vs.yaml
virtualservice.networking.istio.io/reviews replaced
```

2.5. Run the `traffic_gen.py` script to verify the weighted routing configuration.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/rev
iews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/100] ✅ HTTP 200 -- red (213.9ms)
[2/100] ✅ HTTP 200 -- No stars (56.5ms)
[3/100] ✅ HTTP 200 -- No stars (15.3ms)
[4/100] ✅ HTTP 200 -- No stars (10.4ms)
[5/100] ✅ HTTP 200 -- No stars (10.0ms)

...output omitted...

📈 Response Distribution
================================================================================
┌──────────────────────────┬──────────┬──────────────────────┐
│ Response                 │ Count    │ Percentage           │
├──────────────────────────┼──────────┼──────────────────────┤
│ No stars                 │ 81       │          81.0%       │
│ black                    │ 15       │          15.0%       │
│ red                      │ 4        │           4.0%       │
└──────────────────────────┴──────────┴──────────────────────┘
```

Your values might differ.

Notice that the distribution closely matches the configured weights (80/15/5). The virtual service OSSM resource now controls traffic distribution instead of relying on Kubernetes' random load balancing. This demonstrates weighted routing's deterministic control over traffic percentages.

3. Implement header-based routing for user segmentation.

In this step, you implement header-based routing to direct different user types to different service versions based on custom HTTP headers. This technique enables feature flagging and user segmentation without application code changes.

You configure routing rules that inspect the `x-user-type` HTTP header:

Header Value	Routes To	Response	Use Case
`x-user-type: internal`	v3	red stars	Internal employees see the latest features
`x-user-type: beta`	v2	black stars	Beta testers get early access
(no header)	v1	No stars	General public gets the stable version

3.1. Review the `reviews-header-vs.yaml` file. It updates the `reviews` virtual service OSSM resource to route traffic based on HTTP headers, which does the following:

- Routes requests with `x-user-type: internal` header to `reviews-v3`.
- Routes requests with `x-user-type: beta` header to `reviews-v2`.
- Routes all other requests to `reviews-v1` as default.

```
[student@workstation meshtraffic-routing]$ cat reviews-header-vs.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match: (1)
    - uri:
        prefix: /reviews
      headers:
        x-user-type:
          exact: internal
    route:
    - destination:
        host: reviews
        subset: v3
  - match: (2)
    - uri:
        prefix: /reviews
      headers:
        x-user-type:
          exact: beta
    route:
    - destination:
        host: reviews
        subset: v2
  - match: (3)
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        subset: v1
```

(1) Route internal users to v3 (newest features).

(2) Route beta testers to v2.

(3) Default route for all other users to v1 (stable version).

3.2. Update the virtual service OSSM resource.

```
[student@workstation meshtraffic-routing]$ oc replace -f reviews-header-vs.yaml
virtualservice.networking.istio.io/reviews replaced
```

3.3. Try the header-based routing with all the requests without headers.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/rev
iews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/100] ✅ HTTP 200 -- No stars (12.9ms)
[2/100] ✅ HTTP 200 -- No stars (7.9ms)
[3/100] ✅ HTTP 200 -- No stars (6.6ms)
[4/100] ✅ HTTP 200 -- No stars (6.6ms)
[5/100] ✅ HTTP 200 -- No stars (7.1ms)

...output omitted...

📈 Response Distribution
================================================================================

┌──────────────────────────┬──────────┬──────────────────┐
│ Response                 │ Count    │ Percentage       │
├──────────────────────────┼──────────┼──────────────────┤
│ No stars                 │ 100      │        100.0%    │
└──────────────────────────┴──────────┴──────────────────┘
```

All traffic routes to `reviews-v1` (*No stars*) because requests without the `x-user-type` header match the default routing rule. This confirms that header-based routing works correctly for requests without custom headers.

3.4. Verify the `mix.yaml` traffic generator configuration file to send requests with both headers and without headers, mixing the requests.

```
[student@workstation meshtraffic-routing]$ cat mix.yaml
traffic:
  mode: "mix"
...output omitted...
  items:
    - name: "reviews-v1" (1)
      endpoint: "/reviews/1"
      # No headers for v1 (default route)

    - name: "reviews-v2" (2)
      endpoint: "/reviews/1"
      headers:
        default: {"x-user-type": "beta"}

    - name: "reviews-v3" (3)
      endpoint: "/reviews/1"
      headers:
        default: {"x-user-type": "internal"}
...output omitted...
```

(1) Item for `reviews-v1` with no headers, routing to the default version.

(2) Item for `reviews-v2` with `x-user-type: beta` header, routing to the beta version.

(3) Item for `reviews-v3` with `x-user-type: internal` header, routing to the internal version.

3.5. Re-run the `traffic_gen.py` script by using the `mix.yaml` as the configuration file.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py mix.yaml
traffic_gen.py mix.yaml
🧪 Mix mode: pattern=round-robin, items=3
🔗 reviews-v1: curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
🔗 reviews-v2: curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1 -H
'x-user-type: beta'
🔗 reviews-v3: curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1 -H
'x-user-type: internal'
[1] reviews-v1 ✅ HTTP 200 -- No stars (870.2ms)
[2] reviews-v2 ✅ HTTP 200 -- black (1078.6ms)
[3] reviews-v3 ✅ HTTP 200 -- red (1686.7ms)
[4] reviews-v1 ✅ HTTP 200 -- No stars (866.6ms)
[5] reviews-v2 ✅ HTTP 200 -- black (1076.8ms)
[6] reviews-v3 ✅ HTTP 200 -- red (1684.3ms)

...output omitted...

📈 Response Distribution
================================================================================

┌──────────────────────────┬──────────┬──────────────┐
│ Response                 │ Count    │ Percentage   │
├──────────────────────────┼──────────┼──────────────┤
│ No stars                 │ 10       │        33.3% │
│ black                    │ 10       │        33.3% │
│ red                      │ 10       │        33.3% │
└──────────────────────────┴──────────┴──────────────┘
```

The traffic generator `mix` mode sends 30 total requests. Each item receives exactly 10 requests (33.3% distribution). Notice how the script shows the item name (`reviews-v1`, `reviews-v2`, `reviews-v3`) before each result, making it easy to track which routing rule is being tested. The response distribution confirms that header-based routing works correctly.

4. Implement URI path-based routing for API versioning.

In this step, you implement path-based routing to manage API versioning at the service mesh level. This enables you to expose multiple versions of your API simultaneously as you route requests to the appropriate backend version based on the URI path.

You configure routing rules that inspect the request URI:

Request Path	Routes To	Response	API Version
`/api/v3/reviews/*`	v3	red stars	Latest API version
`/api/v2/reviews/*`	v2	black stars	Previous API version
`/reviews/*`	v1	No stars	Legacy/default endpoint

A critical aspect of this configuration is *URI rewriting*. The backend `reviews` service only accepts `/reviews` endpoints, not versioned paths such as `/api/v3/reviews`. OSSM automatically rewrites the incoming URI before forwarding to the backend, transforming `/api/v3/reviews/1` into `/reviews/1`.

4.1. Review the `reviews-path-vs.yaml` file that update the `reviews` virtual service OSSM resource to route traffic based on URI paths:

- Route `/api/v3/*` paths to `reviews-v3`.
- Route `/api/v2/*` paths to `reviews-v2`.
- Route `/api/v1/*` or default paths to `reviews-v1`.

```
[student@workstation meshtraffic-routing]$ cat reviews-path-vs.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match: (1)
    - uri:
        prefix: /api/v3/reviews
    rewrite: (2)
      uri: /reviews
    route:
    - destination:
        host: reviews
        subset: v3
  - match:
    - uri:
        prefix: /api/v2/reviews
    rewrite: (3)
      uri: /reviews
    route:
    - destination:
        host: reviews
        subset: v2
  - match: (4)
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        subset: v1
```

(1) Match requests with the `/api/v3/reviews` URI prefix to route to version 3.

(2) Rewrite the URI from `/api/v3/reviews/*` to `/reviews/*` before forwarding to the backend.

(3) Rewrite the URI from `/api/v2/reviews/*` to `/reviews/*` for version 2 requests.

(4) Default route for standard `/reviews` paths to the stable version v1.

4.2. Update the virtual service OSSM resource.

```
[student@workstation meshtraffic-routing]$ oc replace -f reviews-path-vs.yaml
virtualservice.networking.istio.io/reviews replaced
```

4.3. Confirm that the `finite.yaml` configuration file is sending the requests to the `/reviews` endpoint.

```
[student@workstation meshtraffic-routing]$ cat finite.yaml
traffic:
  mode: "finite"
  endpoint: "/reviews/1"

...output omitted...
```

4.4. Verify that requests to `/reviews` route to `reviews-v1` (No stars). Run the `traffic_gen.py` script by using the `finite.yaml` as the configuration file.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/rev
iews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/100] ✅ HTTP 200 -- No stars (24.8ms)
[2/100] ✅ HTTP 200 -- No stars (7.2ms)
[3/100] ✅ HTTP 200 -- No stars (5.6ms)
[4/100] ✅ HTTP 200 -- No stars (6.7ms)
[5/100] ✅ HTTP 200 -- No stars (6.0ms)

...output omitted...

📈 Response Distribution
================================================================================

┌──────────────────────────┬─────────┬──────────────────────┐
│ Response                 │ Count   │ Percentage           │
├──────────────────────────┼─────────┼──────────────────────┤
│ No stars                 │ 100     │        100.0%        │
└──────────────────────────┴─────────┴──────────────────────┘
```

All traffic routes to `reviews-v1` (*No stars*) because the request path `/reviews` matches the default routing rule. This confirms that path-based routing correctly handles the legacy endpoint.

4.5. Confirm that the `finite-api-v2.yaml` configuration file is sending the requests to the `/api/v2/reviews` endpoint.

```
[student@workstation meshtraffic-routing]$ cat finite-api-v2.yaml
traffic:
  mode: "finite"
  endpoint: "/api/v2/reviews/1"

...output omitted...
```

4.6. Verify that requests to `/api/v2/reviews` route to `reviews-v2` (black stars). Run the `traffic_gen.py` script by using the `finite-api-v2.yaml` as the configuration file.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite-api-v2.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/ap
i/v2/reviews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/api/v2/reviews/1
[1/100] ✅ HTTP 200 -- black (226.0ms)
[2/100] ✅ HTTP 200 -- black (34.0ms)
[3/100] ✅ HTTP 200 -- black (30.7ms)
[4/100] ✅ HTTP 200 -- black (19.5ms)
[5/100] ✅ HTTP 200 -- black (26.6ms)

...output omitted...

📈 Response Distribution
================================================================================

┌──────────────────────────┬─────────┬──────────────────────┐
│ Response                 │ Count   │ Percentage           │
├──────────────────────────┼─────────┼──────────────────────┤
│ black                    │ 100     │        100.0%        │
└──────────────────────────┴─────────┴──────────────────────┘
```

All traffic routes to `reviews-v2` (*black stars*) because the request path `/api/v2/reviews` matches the second routing rule. The virtual service automatically rewrote `/api/v2/reviews/1` to `/reviews/1` before forwarding to the backend. This demonstrates URI rewriting enables API versioning without backend modifications.

4.7. Confirm that the `finite-api-v3.yaml` configuration file is sending the requests to the `/api/v3/reviews` endpoint.

```
[student@workstation meshtraffic-routing]$ cat finite-api-v3.yaml
traffic:
  mode: "finite"
  endpoint: "/api/v3/reviews/1"

...output omitted...
```

4.8. Verify that requests to `/api/v3/reviews` route to `reviews-v3` (red stars). Run the `traffic_gen.py` script by using the `finite-api-v3.yaml` as the configuration file.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite-api-v3.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/ap
i/v3/reviews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/api/v3/reviews/1
[1/100] ✅ HTTP 200 -- red (126.0ms)
[2/100] ✅ HTTP 200 -- red (23.7ms)
[3/100] ✅ HTTP 200 -- red (25.0ms)
[4/100] ✅ HTTP 200 -- red (20.2ms)
[5/100] ✅ HTTP 200 -- red (22.4ms)

...output omitted...

📈 Response Distribution
================================================================================
┌──────────────────────────┬──────────┬──────────────────────┐
│ Response                 │ Count    │ Percentage           │
├──────────────────────────┼──────────┼──────────────────────┤
│ red                      │ 100      │        100.0%        │
└──────────────────────────┴──────────┴──────────────────────┘
```

All traffic routes to `reviews-v3` (*red stars*) because the request path `/api/v3/reviews` matches the first routing rule. OSSM rewrote the URI and routed the request to `v3`, completing the path-based routing demonstration for all three API versions.

5. Implement progressive canary deployment with traffic shifting.

In this step, you simulate a real-world canary deployment scenario where you gradually roll out a new version of the `reviews` service. You start by directing only 5% of traffic to the new version (v3) to minimize risk, then progressively increase to 25% and finally 50% as you gain confidence in the new version's stability.

This approach enables:

- Detect issues early with minimal user impact
- Monitor performance and error rates at each stage
- Roll back quickly if problems arise
- Build confidence before full deployment

5.1. Review the `reviews-canary-5pct-vs.yaml` file that implements a conservative canary rollout with 5% traffic to v3.

```
[student@workstation meshtraffic-routing]$ cat reviews-canary-5pct-vs.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        subset: v1
      weight: 90 (1)
    - destination:
        host: reviews
        subset: v2
      weight: 5
    - destination:
        host: reviews
        subset: v3
      weight: 5 (2)
```

(1) Keep the majority of traffic on the stable version.

(2) Start with only 5% of traffic to the new canary version (v3).

5.2. Update the virtual service OSSM resource.

```
[student@workstation meshtraffic-routing]$ oc replace -f reviews-canary-5pct-vs.yaml
virtualservice.networking.istio.io/reviews replaced
```

5.3. Verify the initial canary deployment to see the distribution.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/rev
iews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/100] ✅ HTTP 200 -- red (60.2ms)
[2/100] ✅ HTTP 200 -- No stars (42.3ms)
[3/100] ✅ HTTP 200 -- No stars (11.9ms)
[4/100] ✅ HTTP 200 -- No stars (8.3ms)
[5/100] ✅ HTTP 200 -- No stars (7.7ms)

...output omitted...


📈 Response Distribution
================================================================================

┌──────────────────────────┬──────────┬──────────────┐
│ Response                 │ Count    │ Percentage   │
├──────────────────────────┼──────────┼──────────────┤
│ No stars                 │ 91       │        91.0% │
│ red                      │ 5        │         5.0% │
│ black                    │ 4        │         4.0% │
└──────────────────────────┴──────────┴──────────────┘
```

Your values might differ.

The canary version (red/v3) receives approximately 5% of traffic as configured. This minimal exposure limits the impact if the new version contains issues. In a production scenario, you would monitor error rates, latency, and business metrics before proceeding to the next stage.

5.4. Progressively increase traffic to 25%. Review the `reviews-canary-25pct-vs.yaml` file.

```
[student@workstation meshtraffic-routing]$ cat reviews-canary-25pct-vs.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        subset: v1
      weight: 70
    - destination:
        host: reviews
        subset: v2
      weight: 5
    - destination:
        host: reviews
        subset: v3
      weight: 25 (1)
```

(1) Increase canary traffic to 25%.

5.5. Update the virtual service OSSM resource.

```
[student@workstation meshtraffic-routing]$ oc replace -f reviews-canary-25pct-vs.yaml
virtualservice.networking.istio.io/reviews replaced
```

5.6. Verify the increased canary traffic.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/rev
iews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/100] ✅ HTTP 200 -- No stars (31.5ms)
[2/100] ✅ HTTP 200 -- red (57.5ms)
[3/100] ✅ HTTP 200 -- No stars (8.9ms)
[4/100] ✅ HTTP 200 -- No stars (9.3ms)
[5/100] ✅ HTTP 200 -- No stars (7.2ms)

...output omitted...

📈 Response Distribution
================================================================================

┌──────────────────────────┬──────────┬──────────────────────┐
│ Response                 │ Count    │ Percentage           │
├──────────────────────────┼──────────┼──────────────────────┤
│ No stars                 │ 72       │         72.0%        │
│ red                      │ 23       │         23.0%        │
│ black                    │ 5        │          5.0%        │
└──────────────────────────┴──────────┴──────────────────────┘
```

Your values might differ.

The canary version (red/v3) receives approximately 25% of traffic, demonstrating the second phase of the progressive rollout. After monitoring shows the canary version performs well at this level, you can confidently increase exposure to 50%.

5.7. Complete the canary rollout by shifting 50% of traffic to v3. Review the `reviews-canary-50pct-vs.yaml` file that balances traffic between stable and canary versions.

```
[student@workstation meshtraffic-routing]$ cat reviews-canary-50pct-vs.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        subset: v1
      weight: 45
    - destination:
        host: reviews
        subset: v2
      weight: 5
    - destination:
        host: reviews
        subset: v3
      weight: 50 (1)
```

(1) The canary version is now receiving half of all traffic.

5.8. Update the virtual service OSSM resource.

```
[student@workstation meshtraffic-routing]$ oc replace -f reviews-canary-50pct-vs.yaml
virtualservice.networking.istio.io/reviews replaced
```

5.9. Verify the final traffic distribution.

```
[student@workstation meshtraffic-routing]$ traffic_gen.py finite.yaml
🎯 Finite mode: 100 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/rev
iews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/100] ✅ HTTP 200 -- No stars (12.8ms)
[2/100] ✅ HTTP 200 -- No stars (7.7ms)
[3/100] ✅ HTTP 200 -- No stars (7.2ms)
[4/100] ✅ HTTP 200 -- red (33.5ms)
[5/100] ✅ HTTP 200 -- red (15.7ms)

...output omitted...
📈 Response Distribution
================================================================================

┌──────────────────────────┬─────────┬──────────────────────┐
│ Response                 │ Count   │ Percentage           │
├──────────────────────────┼─────────┼──────────────────────┤
│ red                      │ 51      │         51.0%        │
│ No stars                 │ 40      │         40.0%        │
│ black                    │ 9       │          9.0%        │
└──────────────────────────┴─────────┴──────────────────────┘
```

Your values might differ.

The canary version (red/v3) now receives approximately 50% of traffic, equal to the stable version. This final phase validates that the new version can handle production-level load before fully promoting it. You have successfully completed a progressive canary deployment strategy from 5% to 25% to 50%. This demonstrates how OSSM enables risk-controlled rollouts through gradual traffic shifting.

Finish

On the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish meshtraffic-routing
```